Kingdom of Saudi Arabia

Ministry of Education

Taibah University (code 039)

College of Computer Science and Engineering

Cybersecurity Department

CYB270 – Cryptography Group Project

" Encryption and Decryption Cipher Tool"

Presented as a requirement for Cryptography CYB 270 courses.

Presented by:

F3

ID	NAME
4450974	Dhay Nasser Al-Harbi
4454418	Layan Faisal Al-Aboudi
4451074	Leen Ahmed Al-Oufi
4453139	Roaa Al-Juhani
4450562	Yara Al-Harbi

Supervised by:

Reem kateb

Second Semester
2025-2026

1. Introduction

This project aims to develop a software tool that enables users to encrypt and decrypt text and text files using various encryption algorithms. The tool is designed to be user-friendly while supplying a proficient level of security for textual data.

Supported algorithms:

- Affine Cipher
- Atbash Cipher
- Data Encryption Standard (DES)

1.1 Cipher Tool

This tool runs in the console and was created with the fundamentals of Java. To use:

Launch the project in any Java integrated development environment (IDE), like NetBeans. Launch the application (for example, by pressing Shift + F6 in NetBeans). You will be able to use the menu-driven CLI once it appears. Swing was used in the Java development of this tool. To put it to use:

1. Launch the project in any IDE that supports Java, such as IntelliJ or NetBeans.
2. Compile and execute the CipherToolFX.java class.
3. The graphical user interface will start up on its own.
4. Further than the JDK, no setup or installation is necessary.

2. Affine Cipher

2.1 Overview of the Affine cipher

The Affine cipher is a type of monoalphabetic substitution cipher, wherein each letter in an alphabet is mapped to its numeric equivalent, encrypted using a simple mathematical function, and converted back to a letter. [1]

2.2 Affine Cipher Algorithm Steps [2]

1. Start with the original text you want to encrypt.
2. Loop through each character in the text, one by one.
3. Check the type of character:

    - If it is an English letter (A–Z or a–z), proceed to the next step.
    - If it is a non-alphabetic character (like numbers, symbols, or spaces), leave it unchanged.

4. Apply the Affine transformation:

    a) For uppercase letters (A–Z):
        1. Substitute the letter with its position in the alphabet (A = 0, B = 1, ..., Z = 25).
        2. Apply the formula:
        Encrypted = (a × position + b) mod 26
        3. Take the resulting number and convert it back into a letter (0 = A, 1 = B, ..., 25 = Z).

    b) For lowercase letters (a–z):

1. Transform the letter to its position (a = 0, ..., z = 25).
2. Apply the same formula:
   Encrypted = (a × position + b) mod 26
3. Translate into a lowercase letter.

5. Replace each letter with its corresponding transformed letter.
6. Combine the resulting characters to form the final encrypted text.
7. To decrypt:

- Find the modular multiplicative inverse of a modulo 26 (denote it as a_inv).
- For every encrypted letter: Translate the letter to its position.
- Use the decryption formula: Decrypted = (a_inv × (position – b)) mod 26.
- Convert the obtained number to its corresponding letter (uppercase or lowercase).

Table 1. example.

Encryption: Key Values a=17, b=20 — Plaintext: 'CYBER'

Original Text	x	ax+b % 26	Encrypted Text
C	2	2	C
Y	24	12	M
B	1	11	L
E	4	10	K
R	17	23	X

Table 2. example.

Decryption: a^{-1} = 23 — Ciphertext: 'CMLKX'

Encrypted Text	Encrypted Valu	23*(x-b) mod 26	Decrypted Text
C	2	2	C
M	12	24	Y
L	11	1	B
K	10	4	E
X	23	17	R

2.3 Interface and Usage of the Affine Cipher within the Tool

1. Choose Cipher: From the "Cipher" dropdown, select affine.
2. Enter a File Path (Optional): In the "File Path" field, enter the path to a .txt file if you want to load text from a file.
3. Enter The First Key: In the “Key a” field, pick a number that is a co-prime with 26.
4. Enter The Second Key: In the “Key b” field, enter any whole number.
5. Select Operation: Use the radio buttons to choose “Encrypt” or “Decrypt”.
6. Click “Process”: This will perform the selected operation on either the loaded file content or the manually entered text.
7. View Output: The result will appear in the “Output” field at the bottom of the screen.
8. Clear Output (Optional): Use the “Clear” button to reset the input and output fields.

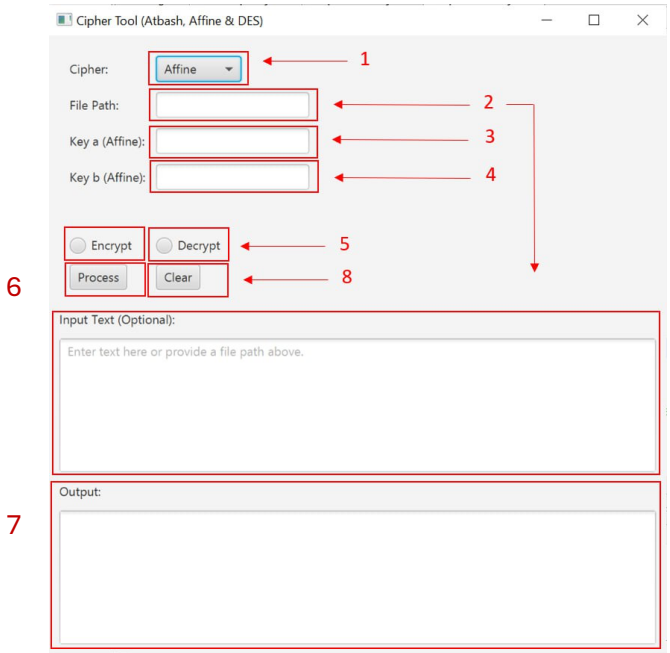

Figure 1. Graphical user interface of the Cipher Tool

2.4 Example

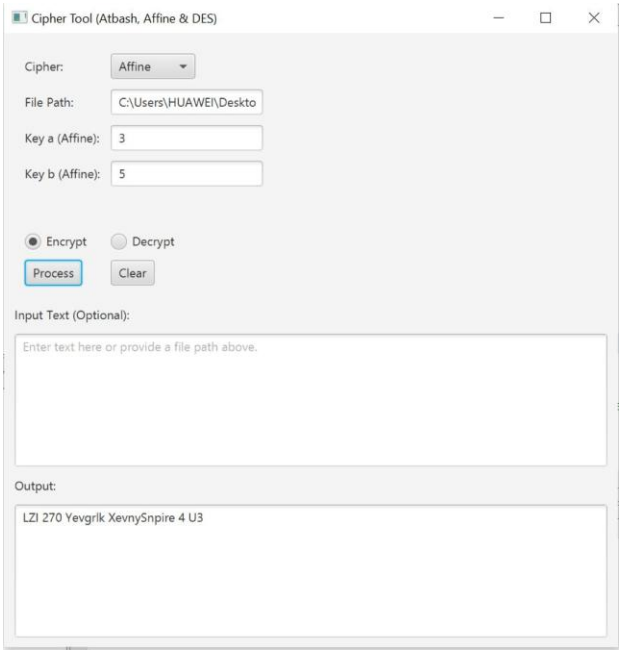

Figure 2. An example of encryption.

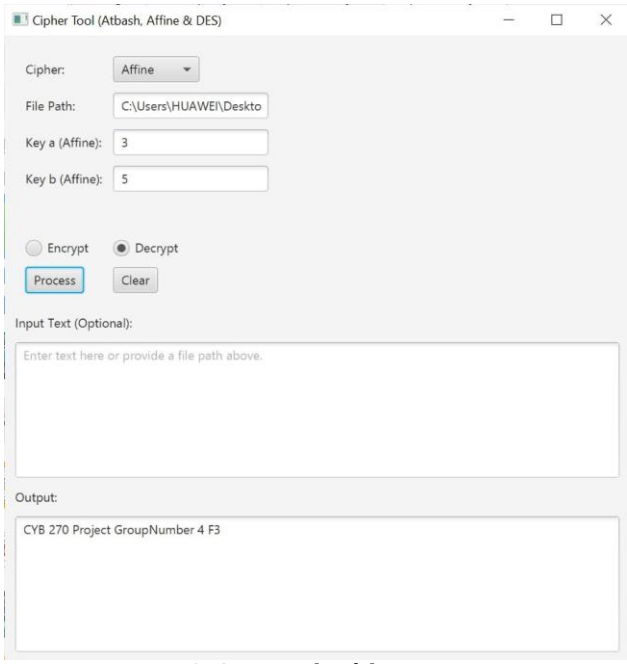

Figure 3. An example of decryption.

2.5 Troubleshooting

- Invalid Key Value:
  - Problem: Entering a value for 'a' which is not coprime to 26 may result in failure or incorrect results for the decryption process.
  - Solution: Choose a value for 'a' which is co-prime with 26.
- File Not Found:
  - Problem: The file path entered does not lead to a valid .txt file.
  - Solution: Double-check for spelling errors, use the full absolute path, and ensure the file actually exists.
- No Output Appears:
  - Problem: After clicking the “Process” button, no output is displayed.
  - Solution: Make sure the file contains readable text and that an operation (Encrypt or Decrypt) is selected.
- Unexpected Error Message:
  - Problem: The program displays a Java exception or crashes.
  - Solution: Ensure all the necessary fields are filled in, the file is not in use or locked by another program, and that it's readable.

3. Atbash Cipher

3.1 Overview of the Atbash cipher

The Atbash Cipher is a really simple substitution cipher that is sometimes called mirror code. It is believed to be the first cipher ever used, and its use pre-dates Egyptian examples of encryption. To use Atbash, you simply reverse the alphabet, so A encodes to Z, B to Y and so on.

Atbash is considered a special case of Affine Cipher, a monoalphabetic substitution cipher. Affine is encrypted by converting letters to their numerical equivalent (A=1, Z=26 etc.), putting that number through a mathematical formula, and the converting the result into letters. With Atbash, the Affine formula is a = b = (m − 1), where m is the length of the alphabet. [3]

3.2 Atbash Cipher Algorithm Steps

Atbash encryption uses a substitution alphabet and its reciprocal, a combination of the normal alphabet and its reverse alphabet (mirrored). [3]

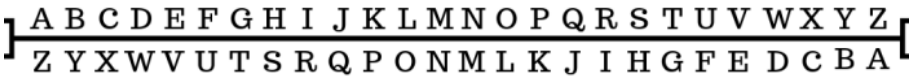

Figure 4. Atbash Cipher translation [3].

1. Start with the original text you want to encrypt.
2. Loop through each character in the text, one by one.
3. Check the type of character:

   - If it is an English letter (A–Z or a–z), proceed to the next step.
   - If it is a non-alphabetic character (like numbers, symbols, or spaces), leave it unchanged.

4. Reverse the position of the letter in the alphabet:

   - For uppercase letters (A–Z): Encrypted=′Z′−(Letter−′A′)
   - For lowercase letters (a–z): Encrypted=′z′−(Letter−′a′)

5. Replace each letter with its corresponding letter in the reversed alphabet.
6. Combine the resulting characters to form the final encrypted text.
7. To decrypt: Simply repeat the same steps on the encrypted text (since Atbash is symmetric). [4]

3.3 Interface and Usage of the Atbash Cipher within the Tool

1. Choose Cipher: From the "Cipher" dropdown, select Atbash.
2. Enter Input: Either in the “Input Text (Optional)” field as a text, or in the "File Path" field, enter the path to a .txt file if you want to load text from a file.
3. Select Operation: Use the radio buttons to choose “Encrypt” or “Decrypt”.
4. Click “Process”: This will perform the selected operation on either the loaded file content or the manually entered text.
5. View Output: The result will appear in the “Output” field at the bottom of the screen.
6. Clear Output (Optional): Use the “Clear” button to reset the input and output fields.

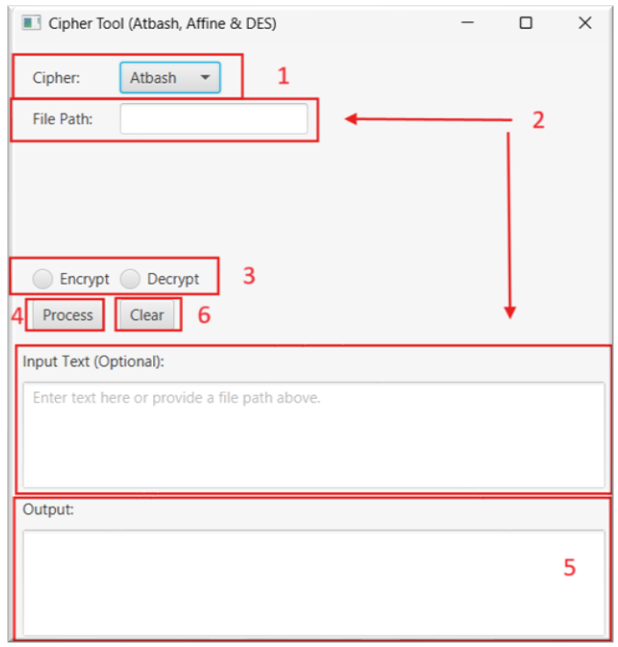

Figure 5. Graphical user interface of the Cipher Tool

3.4 Example

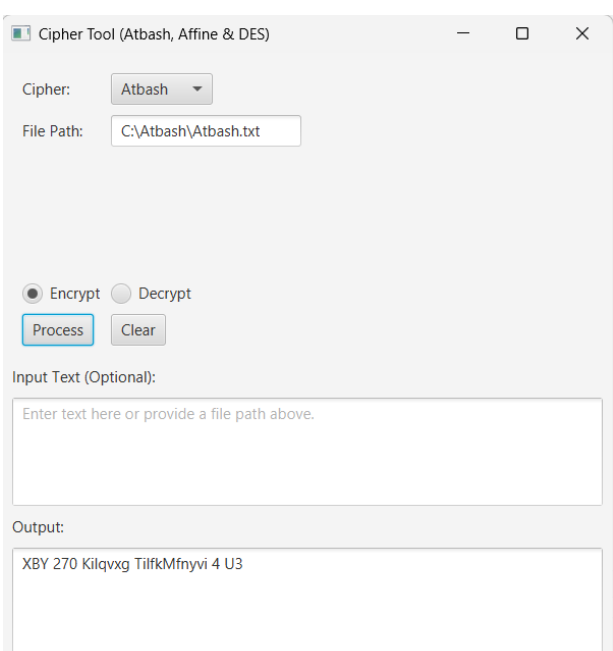

Figure 6. An example of encryption.

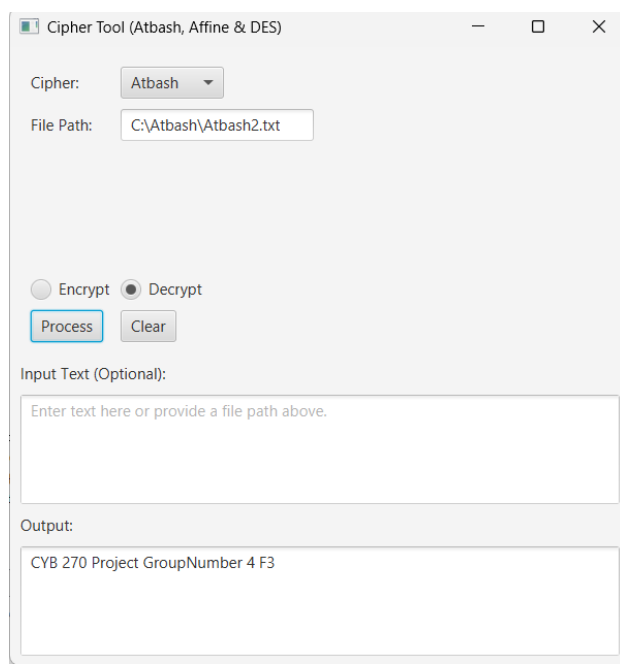

Figure 7. An example of decryption.

3.5 Troubleshooting

- File Not Found:
  - Problem: The file path entered does not lead to a valid .txt file.
  - Solution: Double-check for spelling errors, use the full absolute path, and ensure the file actually exists.
- No Output Appears:
  - Problem: After clicking the “Process” button, no output is displayed.
  - Solution: Make sure the file contains readable text and that an operation (Encrypt or Decrypt) is selected.
- Unexpected Error Message:
  - Problem: The program displays a Java exception or crashes.
  - Solution: Ensure all the necessary fields are filled in, the file is not in use or locked by another program, and that it's readable.

4. Data Encryption Standard DES

4.1 Overview of the DES Cipher

The Data Encryption Standard (DES) is a symmetric-key block cipher that was developed in the early 1970s and later adopted as a federal standard by NIST. DES encrypts data by dividing it into fixed-size blocks and applying a series of transformations using a secret key. It was one of the first widely accepted encryption standards used to protect sensitive information. [5]

DES uses a 64-bit key, but only 56 bits are actually used for encryption; the remaining 8 bits are used for error checking (parity). Despite its historical importance, DES is now considered insecure for modern applications because of its short key length. However, it is still studied as a foundation for understanding modern encryption. [6]

4.2 DES Cipher Algorithm Steps [7]

The DES algorithm processes data in blocks of 64 bits using a series of well-defined steps. The figure above illustrates the internal structure of DES. Here’s how it works:

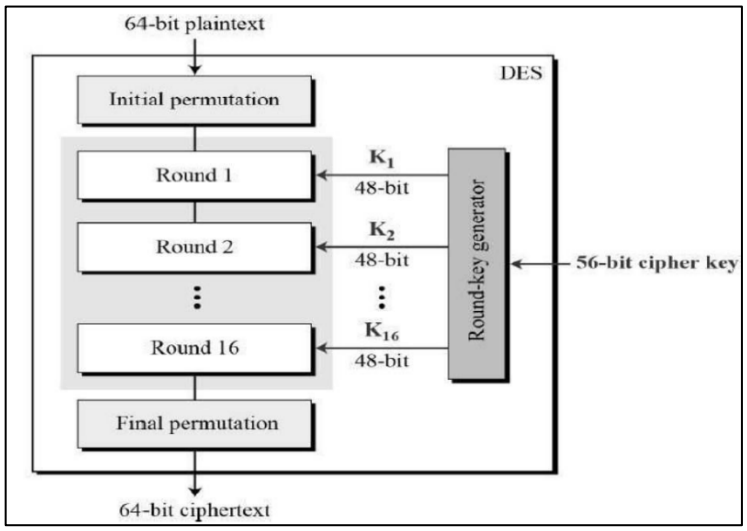

Figure 8. Block diagram [7].

1. Input:
   - The algorithm takes a 64-bit block of plaintext as input.
   - It also takes a 56-bit cipher key (usually entered as 8 ASCII characters — 64 bits, where 8 bits are parity and not used in encryption).
2. Initial Permutation (IP): Before the encryption rounds begin, the 64-bit plaintext block goes through an initial permutation that rearranges the bits using a fixed table.
3. Round-Key Generator:
   - The 56-bit cipher key is used to generate 16 different 48-bit subkeys (K_1 to K_{16}), one for each row.
   - This is done using left-shifting and permutation functions.
4. 16 Encryption Rounds: Each round takes the output of the previous round and applies:

A. Expansion: Expands 32 bits to 48 bits.
B. Key mixing: The expanded bits are XORed with the round key.
C. Substitution: The result is passed through 8 S-boxes to reduce it back to 32 bits.
D. Permutation: The 32 bits are rearranged using a P-box.

The result is XORed with the other half of the block (Feistel structure), and the two halves are swapped.

5. Final Permutation (FP): After all 16 rounds, the halves are swapped once more, and the combined block goes through a final permutation to produce the ciphertext.
6. Output: The result is a 64-bit encrypted block (ciphertext).

4.3 Interface and Usage of the DES Cipher within the Tool

1. Choose Cipher: From the "Cipher" dropdown, select DES.
2. Enter Input: Either in the “Input Text (Optional)” field as a text, or in the "File Path" field, enter the path to a .txt file if you want to load text from a file.
3. Enter the Key: In the "Key (DES)" field, enter a string of exactly 8 characters. *(DES requires a 64-bit key, represented by 8 characters)*
4. Select Operation: Use the radio buttons to choose “Encrypt” or “Decrypt”.
5. Click “Process”: This will perform the selected operation on either the loaded file content or the manually entered text.
6. View Output: The result will appear in the “Output” field at the bottom of the screen.
7. Clear Output (Optional): Use the “Clear” button to reset the input and output fields.

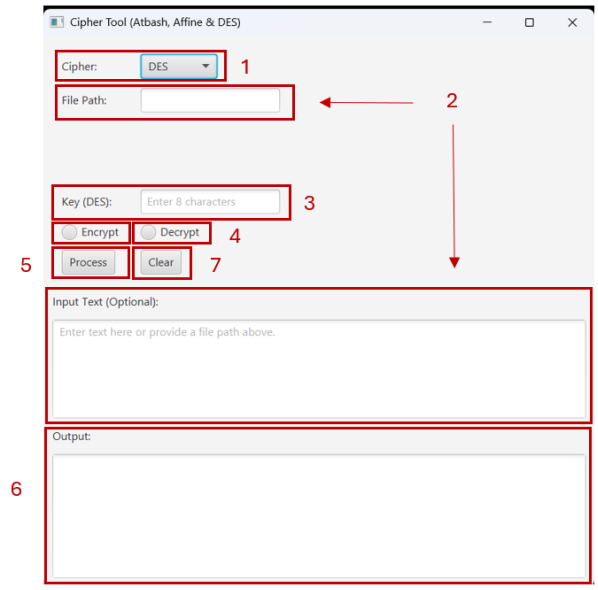

Figure 9. Graphical user interface of the Cipher Tool.

4.4 Example

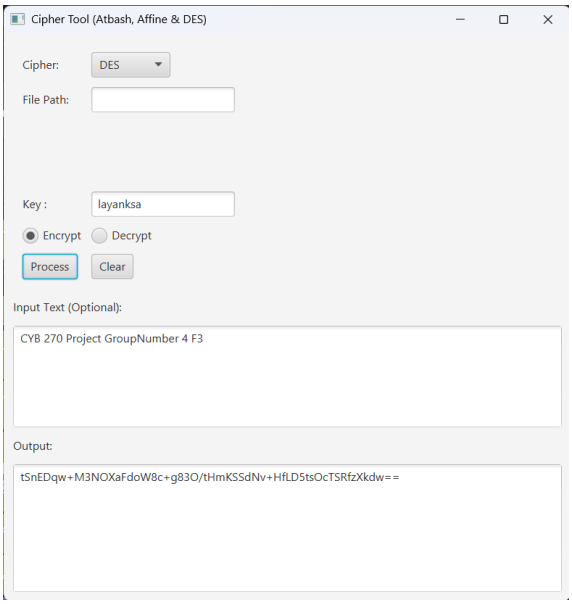

Figure 10. An example of encryption.

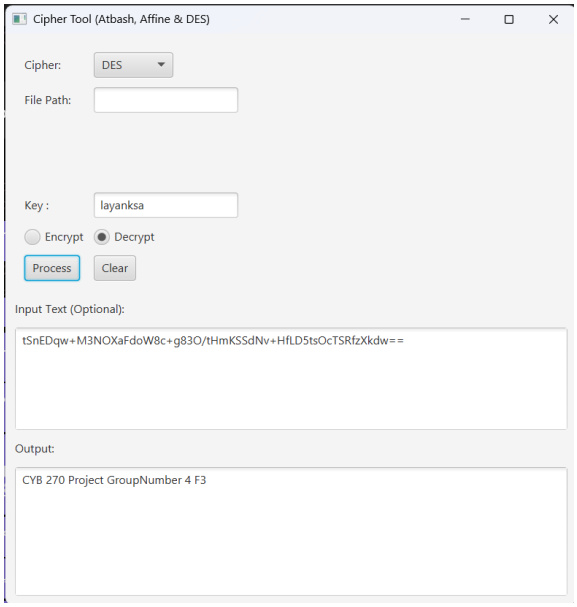

Figure 11. An example of decryption.

Important Notes:

- The key must be exactly 8 characters.
- This tool uses the ECB (Electronic Codebook) mode by default.
- Padding is handled automatically using PKCS5Padding.

4.5 Troubleshooting

- Invalid Key Length:
  - Problem: Entering a key that is not exactly 8 characters.
  - Solution: Make sure the key is exactly 8 characters long.
- Input is Not Base64 During Decryption:
  - Problem: Trying to decrypt plain text instead of encrypted Base64 text.
  - Solution: Only decrypt data that was previously encrypted using DES and encoded with Base64.
- File Not Found:
  - Problem: The file path entered is incorrect.
  - Solution: Double-check the file path and make sure the file exists.

5. Conclusion

This project bridges mathematical foundations (Affine Cipher), historical encryption techniques (Atbash Cipher), and modern standards (DES) into a single educational tool. Key achievements include:
Hands-on Learning: Simplified experimentation with substitution ciphers and symmetric-key encryption.
Algorithm Comparison: Highlighted the evolution of security (e.g., DES's 56-bit key vs. Atbash's vulnerability).
Dual-Language Access: Promoted inclusivity in understanding cryptography's global impact.
The Big Question:
"How can emerging technologies—like quantum computing or AI—reshape the balance between historical ciphers and future encryption needs?"
This project invites students and researchers to explore this challenge, paving the way for innovations in cross-era cryptographic design.

6. Resources

[1] GeeksforGeeks, "Implementation of Affine Cipher." [Online]. Available: https://www.geeksforgeeks.org/implementation-affine-cipher/
[2] Crypto Corner, "Affine Cipher – Interactive Tool." [Online]. Available: https://crypto.interactive-maths.com/affine-cipher.html
[3] Boxentriq, "Atbash Cipher (Online Tool)." [Online]. Available: https://www.boxentriq.com/code-breaking/atbash-cipher
[4] Dcode, "Atbash Cipher – Backwards/Reverse Alphabet Decoder." [Online]. Available: https://www.dcode.fr/atbash-cipher
[5] B. Schneier, Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd ed., John Wiley & Sons, 2015.
[6] National Institute of Standards and Technology (NIST), Data Encryption Standard (DES). [Online]. Available: https://csrc.nist.gov/publications/detail/fips/46-3/final
[7] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed., Pearson Education, 2017.

7. Additional Resources

For those interested in further exploring cryptography, we recommend the following authoritative resources:

- B. Schneier, Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd ed., John Wiley & Sons, 2015.
- J. Katz and Y. Lindell, Introduction to Modern Cryptography, 2nd ed., CRC Press, 2020.
- Boneh, Cryptography I, Coursera. [Online]. Available: https://www.coursera.org/learn/crypto
- OWASP, Cryptographic Storage Cheat Sheet. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- Boxentriq, "Atbash Cipher," [Online]. Available: https://www.boxentriq.com/code-breaking/atbash-cipher

These resources supply both theoretical foundations and practical implementations to deepen your understanding of cryptographic systems.